

Fase 1 - Orientação a Objetos

Esse material cobre herança, abstração, interface, sobrecarga e sobrescrição. Leia cada explicação com calma, teste o código de exemplo no seu computador (não só leia - roda mesmo) e só depois vá para o exercício daquele tópico.

1. Herança

O que é: Herança é o mecanismo que permite uma classe (subclasse/filha) reaproveitar atributos e métodos de outra classe (superclasse/mãe). Serve pra evitar repetir código quando várias classes compartilham um comportamento em comum.

Exemplo:

```
class Animal {
    String nome;

    void emitirSom() {
        System.out.println("Som generico");
    }
}

class Cachorro extends Animal {
    void emitirSom() {
        System.out.println("Latido");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal meuAnimal = new Cachorro();
        meuAnimal.emitirSom(); // imprime "Latido"
    }
}
```

Ponto de atenção: Repare que a variável é do tipo `Animal`, mas o objeto criado é um `Cachorro`. Quando você chama `emitirSom()`, quem responde é o método do `Cachorro`, não do `Animal`. Isso é polimorfismo - o comportamento é decidido pelo tipo real do objeto em tempo de execução, não pelo tipo da variável.

Java **não** permite herança múltipla de classes (uma classe só pode estender uma única classe). Isso existe pra evitar ambiguidade: se duas superclasses tivessem um método com o mesmo nome e comportamentos diferentes, o Java não saberia qual usar.

Exercício 1.1 Crie uma classe abstrata (por enquanto trate só como uma classe normal) chamada `Funcionario`, com atributo `nome` e um método `calcularSalario()` que retorna um valor fixo qualquer. Crie duas subclasses: `Gerente` e `Vendedor`, cada uma sobrescrevendo `calcularSalario()` com uma lógica diferente (pode ser um valor fixo diferente por enquanto). No `main`, crie uma lista de `Funcionario` contendo um `Gerente` e um `Vendedor`, percorra a lista e chame `calcularSalario()` de cada um. Explique com suas palavras por que cada um imprime um valor diferente mesmo chamando o mesmo método.

2. Abstração (classes abstratas)

O que é: Uma classe abstrata é uma classe que não pode ser instanciada diretamente. Ela serve como um “molde incompleto” - define coisas que as subclasses têm em comum, mas deixa alguns métodos sem implementação (métodos abstratos), obrigando cada subclasse a implementar do seu jeito.

Exemplo:

```
abstract class Forma {
    abstract double calcularArea(); // sem corpo - cada subclasse implementa

    void imprimirArea() {
        System.out.println("Area: " + calcularArea());
    }
}

class Circulo extends Forma {
    double raio;

    Circulo(double raio) {
        this.raio = raio;
    }

    double calcularArea() {
        return Math.PI * raio * raio;
    }
}

class Retangulo extends Forma {
    double largura, altura;

    Retangulo(double largura, double altura) {
        this.largura = largura;
        this.altura = altura;
    }

    double calcularArea() {
        return largura * altura;
    }
}
```

Repare: `new Forma()` daria erro de compilação. Só dá pra instanciar `Circulo` ou `Retangulo`, que são concretas.

Quando usar classe abstrata em vez de interface: Use classe abstrata quando as subclasses compartilham estado (atributos) e/ou código pronto em comum, além do contrato. Use interface quando você só quer garantir que uma classe “sabe fazer” algo, sem se importar com como ela é estruturada por dentro (e principalmente quando classes completamente diferentes precisam ter esse comportamento em comum).

Exercício 2.1 Transforme a classe `Funcionario` do exercício anterior em uma classe abstrata, com o método `calcularSalario()` sendo abstrato (sem corpo). Tente instanciar `new Funcionario()` no main e observe o erro que o Java te dá. Cole o erro e explique com suas palavras por que ele acontece.

3. Interface

O que é: Uma interface define um contrato: um conjunto de métodos que qualquer classe que “assinar” esse contrato (implements) é obrigada a implementar. Diferente de classe abstrata, interface não guarda estado (não tem atributos com valor, só constantes) e uma classe pode implementar **várias** interfaces ao mesmo tempo (isso resolve o problema da herança múltipla que vimos no tópico 1).

Exemplo:

```
interface Descontavel {
    double aplicarDesconto(double percentual);
}

interface Taxavel {
    double calcularImposto();
}

class Produto implements Descontavel, Taxavel {
    double preco;

    Produto(double preco) {
        this.preco = preco;
    }

    public double aplicarDesconto(double percentual) {
        return preco - (preco * percentual / 100);
    }

    public double calcularImposto() {
        return preco * 0.1;
    }
}
```

Métodos default (desde o Java 8): Interfaces podem ter métodos com corpo, marcados com `default`. Isso foi criado pra permitir evoluir uma interface (adicionar método novo) sem quebrar todas as classes que já implementavam ela antes - se a classe não

sobrescrever o método default, ela usa a implementação padrão da interface.

```
interface Descontavel {
    double aplicarDesconto(double percentual);

    default double aplicarDescontoMaximo() {
        return aplicarDesconto(50); // usa o metodo acima
    }
}
```

Exercício 3.1 Pegue a classe Produto do exemplo acima e crie uma terceira interface chamada Auditavel, com um método default gerarLogAuditoria() que apenas imprime “Produto auditado” no console (não precisa ser abstrato, já entrega pronto). Faça Produto implementar essa interface também, sem precisar escrever o corpo do método (aproveitando o default). No main, chame gerarLogAuditoria() num objeto Produto e confirme que funciona mesmo sem você ter implementado nada daquele método na classe.

4. Sobrecarga de métodos (overload)

O que é: Sobrecarga acontece quando você cria vários métodos com o **mesmo nome** na mesma classe, mas com parâmetros diferentes (quantidade e/ou tipo). O compilador decide qual método chamar de acordo com os argumentos que você passa na chamada.

Exemplo:

```
class Calculadora {
    int somar(int a, int b) {
        return a + b;
    }

    double somar(double a, double b) {
        return a + b;
    }

    int somar(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        System.out.println(calc.somar(2, 3));           // usa o primeiro
        System.out.println(calc.somar(2.5, 3.5));       // usa o segundo
        System.out.println(calc.somar(1, 2, 3));        // usa o terceiro
    }
}
```

Ponto de atenção (o erro mais comum): O tipo de retorno sozinho **não** diferencia sobrecarga. Isso aqui não compila, porque pro Java os dois métodos têm a mesma assi-

natura:

```
int calcular(int a, int b) { return a + b; }  
double calcular(int a, int b) { return a + b; } // ERRO - mesma assinatura
```

O que diferencia sobrecarga é sempre a lista de parâmetros (tipo, quantidade ou ordem), nunca o retorno sozinho.

Exercício 4.1 Crie uma classe Validador com três métodos sobrecarregados chamados validar: - validar(String texto) → retorna true se o texto não estiver vazio - validar(int numero) → retorna true se o número for maior que zero - validar(String texto, int tamanhoMinimo) → retorna true se o texto tiver pelo menos aquele tamanho mínimo de caracteres

Chame os três no main passando argumentos diferentes e confirme que o Java escolhe o método certo sozinho, sem você precisar indicar qual usar.

5. Sobrescrição de métodos (override)

O que é: Sobrescrição acontece quando uma **subclasse** redefine um método que já existe na **superclasse**, com a mesma assinatura (mesmo nome, mesmos parâmetros). É o mecanismo que já apareceu no exemplo de herança (Animal/Cachorro) lá no tópico 1.

Diferença entre overload e override (essa é a confusão mais comum): - **Overload (sobrecarga):** mesmo nome, parâmetros diferentes, na mesma classe (ou em classes sem relação de herança). - **Override (sobrescrição):** mesmo nome, mesmos parâmetros, entre superclasse e subclasse.

Exemplo:

```
class Funcionario {  
    double calcularBonus() {  
        return 100.0;  
    }  
}  
  
class Gerente extends Funcionario {  
    @Override  
    double calcularBonus() {  
        return 500.0;  
    }  
}
```

A anotação @Override: Ela não é obrigatória pro código funcionar, mas é fortemente recomendada. Ela avisa o compilador “eu quero sobrescrever um método da superclasse aqui” - se você errar a assinatura (por exemplo, digitar o nome errado ou mudar um parâmetro sem querer), o compilador acusa erro na hora, em vez de você criar um método novo sem perceber e ficar horas debugando por que a “sobrescrição” não está funcionando.

Exemplo do erro que @Override evita:

```

class Funcionario {
    double calcularBonus() {
        return 100.0;
    }
}

class Gerente extends Funcionario {
    @Override
    double calcularBonos() { // erro de digitacao no nome
        return 500.0;
    }
}
// Sem @Override, isso compilaria normalmente e criaria um metodo novo
// (calcularBonos) sem sobrescrever nada - bug silencioso.
// Com @Override, o compilador acusa erro imediatamente.

```

Exercício 5.1 Pegue a classe Funcionario (com herança para Gerente e Vendedor, dos exercícios 1.1/2.1) e adicione o método calcularBonus() na superclasse com um valor genérico. Sobrescreva esse método em Gerente e em Vendedor, cada um com uma regra diferente, sempre usando @Override. Depois, digite errado de propósito o nome do método em uma das subclasses (sem usar @Override) e rode o código - ele vai compilar sem erro, mas o método não vai fazer o que deveria. Agora adicione @Override em cima desse método com nome errado e veja o compilador acusar o erro. Escreva 2-3 linhas explicando o que aconteceu.

Desafio final da Fase 1 (integrando tudo)

Modele um pequeno sistema de Funcionários de uma empresa:

1. Crie uma classe abstrata Funcionario com atributo nome e método abstrato calcularSalario().
2. Crie uma interface Bonificavel com o método calcularBonus() e um método default calcularBonusAnual() que retorna calcularBonus() * 12.
3. Crie as classes Gerente e Vendedor, ambas extends Funcionario e implements Bonificavel, cada uma com sua própria lógica de salário e bônus.
4. Crie dois métodos sobrecarregados chamados exibirResumo: um que recebe só o Funcionario, e outro que recebe o Funcionario mais um booleano indicando se deve incluir o bônus anual no resumo.
5. No main, crie uma lista de Funcionario com um Gerente e um Vendedor, percorra a lista chamando exibirResumo (das duas formas) para cada um.

Ao terminar, escreva um resumo curto (5-6 linhas) contando com suas palavras onde entrou herança, abstração, interface, sobrecarga e sobrescrição nesse exercício. Isso ajuda a fixar o “porquê”, não só o “como”.